

Imagedata Augmentation and Image Classification

Shreya Dey

**B.Tech in Computer Science and Engineering,
Kalinga Institute of Industrial Technology**

Period of Internship: 18th May 2026 – 14th June 2026

Report submitted to: IDEAS – Institute of Data Engineering, Analytics
and Science Foundation, ISI Kolkata

1. Abstract

This project aims to provide a foundational understanding of image processing and machine learning through the use of Python. Basic image manipulation functions like greyscaling, shifting, resizing, and rotating the image have been performed using OpenCV. Images transformation pipelines, including a dataset loader for the Cat and Dog images were built using PyTorch and Torchvision. The cat-Dog image dataset was loaded using ImageFolder and DataLoader utility for better batch processing. Finally, the Digits dataset for handwritten digits, provided in Scikit-Learn was processed, K-Means clustering applied to partition the images into ten distinct clusters that correspond to each digit from 0 to 9. The clustering performance was then evaluated using macro-averaged F1 Score after appropriate label mapping using majority voting. This project explores integration of fundamental image processing, and machine learning tasks using deep learning data loading mechanism, with a demonstration of unsupervised clustering on digit data.

2. Introduction

Image processing and machine learning are fundamental fields in Artificial Intelligence and Data Science. Image processing enables computer systems to process, analyze, and manipulate visual information, whereas machine learning helps them learn hidden patterns and relationships from data.

This project involved a practical approach to these fields, where standard image processing operations like image loading, converting image to grayscale, translation, resizing, and rotating were performed using the OpenCV library. For efficient image data management and transformation for training purposes, PyTorch and Torchvision have been used to create an image transformation pipeline and a data loader. Moreover, K-Means clustering has been applied on handwritten digits data that is available in Scikit-Learn to demonstrate an unsupervised learning approach.

The project gives a hand-on experience of a common computer vision workflow and unifies image processing, data processing in a deep learning framework, and unsupervised clustering methods. Image processing, data loading and unsupervised learning are essential components to many other sophisticated applications like object detection, classification, facial recognition, etc.

Topics Covered During Training

1. Python Programming Fundamentals
2. NumPy for Numerical Computing
3. Data Visualization Concepts
4. Machine Learning Fundamentals
5. Unsupervised Learning Techniques
6. K-Means Clustering
7. Torchvision and Data Augmentation
8. Dataset Preparation and Data Loaders

3. Project Objective

- 1) To do image processing operations such as filtering.
- 2) To know about preprocessing techniques in computer vision.
- 3) To build a transformation pipeline of images with Torchvision.
- 4) To do effective handling of image dataset with ImageFolder and DataLoader.
- 5) To do K-Means clustering on handwritten digit data.
- 6) To calculate the F1 Score and evaluate the clustering quality.
- 7) To have hands-on experience in machine learning and computer vision libraries.

4. Methodology

Step 1: Image Loading

The image "moon-pexels-frank-cone.jpg" was loaded using OpenCV.

```
original_image = cv2.imread("moon-pexels-frank-cone.jpg")
```

Image Shape:

(800, 640, 3)

Step 2: Grayscale Conversion

The RGB image was converted into a grayscale image using OpenCV.

```
grayscale_image = cv2.cvtColor(original_image, cv2.COLOR_BGR2GRAY)
```

Resulting Shape:

(800, 640)

Step 3: Saving Processed Image

The grayscale image was saved to disk.

```
cv2.imwrite("graymoon.jpg", grayscale_image)
```

Step 4: Image Translation

A transformation matrix was created to shift the image:

50 pixels right

100 pixels down

```
M = np.float32([[1,0,50], [0,1,100]])
```

The image was translated using:

```
cv2.warpAffine()
```

Output Shape:

(800, 640, 3)

Step 5: Image Resizing

The image dimensions were changed to:

Width = 150 pixels

Height = 100 pixels

Using:

```
cv2.resize()
```

Output Shape:

(100,150,3)

Step 6: Image Rotation

The image was rotated 90° counter-clockwise around its center.

```
cv2.getRotationMatrix2D()
```

```
cv2.warpAffine()
```

Output Shape:

(640,800,3)

Step 7: Image Transformation Pipeline

A Torchvision transformation pipeline was created:

```
transforms.Compose([
    transforms.Resize((255,255)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor()
])
```

Operations performed:

Resize image

Random horizontal flip

Convert image to tensor

Step 8: Dataset Creation

The Cat-Dog image dataset was loaded using ImageFolder.

```
train_dataset =  
datasets.ImageFolder( root=data_dir,  
    transform=train_transform)
```

Total Images:

22500

Step 9: DataLoader Construction

A DataLoader was created for mini-batch processing.

```
train_loader =  
DataLoader(train_dataset, batch_size=64, shuffle=True)
```

Batch Output:

Images : [64, 3, 224, 224]
Labels : [64]

Step 10: Loading Digits Dataset

The Digits dataset was loaded using:

```
from sklearn.datasets import load_digits
```

Dataset Shape:

Variable	Shape
X	(1797,64)
y	(1797)

Step 11: K-Means Clustering

The clustering model was initialized with:

```
KMeans( n_clusters=10, random_state=42)
```

The model was fitted on feature matrix X.

```
kmeans.fit_predict(X)
```

Ten clusters were created corresponding to digits 0–9.

Step 12: Performance Evaluation

Since cluster labels are arbitrary, majority voting was used to map cluster labels to true labels.

The macro-averaged F1 score was then computed.

```
f1_score( y, mapped_labels, average='macro')
```

5. Data Analysis and Results

Image Processing Results

Operation	Output Shape
Original Image	(800,640,3)
Grayscale Image	(800,640)
Shifted Image	(800,640,3)
Resized Image	(100,150,3)
Rotated Image	(640,800,3)

Dataset Results

Parameter	Value
Total Training Images	22500
Batch Size	64

Digits Dataset Statistics

Parameter	Value
Samples	1797
Features	64
Classes	10

5.1. Machine Learning Analysis

5.1.1 K-Means Clustering

K-Means clustering was applied to group the handwritten digit images into ten clusters.

Model Configuration

Parameter	Value
Algorithm	K-Means
Number of Clusters	10
Random State	42
Learning Type	Unsupervised

The model assigned cluster labels to all 1797 samples.

5.1.2 Cluster Label Mapping

Since cluster labels generated by K-Means do not directly correspond to digit labels, majority voting was used to map each cluster to its most frequent true digit label.

This mapping enabled direct comparison between predicted cluster assignments and actual digit classes.

5.1.3 Model Evaluation

The clustering performance was evaluated using the Macro-Averaged F1 Score.

Evaluation Metric	Value
Macro F1 Score	0.8628

Interpretation

The F1 Score of 0.8628 indicates strong clustering performance. The clusters formed by K-Means were able to capture the underlying structure of the handwritten digit dataset with high accuracy despite the absence of class labels during training.

5.2 Inferential Analysis

For this project no inferential statistical analysis or hypothesis testing was completed due to the objective of this project to exhibit image processing techniques and assess an unsupervised learning model. For this analysis the following tasks were discussed and evaluated:

- Image transformation and pre-processing
- Dataset preparation using Torchvision
- Unsupervised clustering using K-Means
- Model evaluation using F1 Score metric

It was therefore deemed adequate that only the descriptive analysis and metrics for the model were necessary to evaluate the success of this project.

6. Conclusion

In this project, several image processing and machine learning techniques have been shown using Python libraries like OpenCV, Torchvision, PyTorch, and Scikit-Learn. The basic image transformation operations, like converting to grayscale, translation, resizing and rotation have all been implemented. A full image preprocessing pipeline has been developed to prepare images. Images have been prepared, loaded and made available for use with ImageFolder and DataLoader. The dataset of Cat and Dog images has been properly prepared and used for K-Means Clustering. Clustering was done for Digits dataset and achieved macro-averaged F1 score of 0.8628 after using labels. The project helped to implement Computer Vision pipeline, images preparation and also an unsupervised learning technique. Further study can involve the use of supervised learning models like CNN.

7. Appendices

- 1) Scikit-Learn Documentation
<https://scikit-learn.org/>
- 2) Torchvision Documentation
<https://pytorch.org/vision/stable/>
- 3) Github Repository Link:
<https://github.com/shreya1792-pixel/IDEAS-TIH-PROJECT-.git>